



Template de Prompts de IA para Migração do .NET 4.8 para .NET 8

Template de Prompts de IA para Migração do .NET 4.8 para .NET 8

Template de Prompts de IA para Migração do .NET 4.8 para .NET 8

Este documento tem como finalidade descrever, de forma estruturada, o processo adotado para a migração do módulo de administração do ConfigIT para .NET 8.

Estão incluídos os prompts utilizados com ferramentas de IA, bem como os modelos de código aplicados durante a execução. O conteúdo aqui apresentado visa servir como referência oficial para apoiar equipes de desenvolvimento na condução de iniciativas de modernização tecnológica de seus sistemas.

Sumário

1. Preparação e Setup Inicial
2. Migração da Camada Domain
3. Migração da Camada Repository
4. Migração da Camada Aplicação / API
5. Migração da Camada UI Web
6. Remoções e Limpeza
7. Checklist Final e Pós-Migração
8. Arquivo de Contexto para IA ([migration-context.md](#))
9. PowerPrompt: migração em lote de módulos
10. Troubleshooting: erros comuns e correções
11. Apêndice: scripts PowerShell úteis

Observação sobre caminhos e URLs: os caminhos do ambiente foram substituídos por placeholders para facilitar o reuso e evitar exposição de dados sensíveis. Substitua os placeholders antes de executar os passos no seu ambiente:

- `<NEW_SYSTEM_NAME>` → Nome do projeto (usado para definir namespace e client_id)
- `<OLD_SOLUTION_PATH>` → raiz do código legado (.NET 4.8)
- `<NEW_SOLUTION_PATH>` → raiz da nova solução (.NET 8)



- `<NEW_ENTITIES_PATH>` , `<NEW_MAPPING_PATH>` , etc. → caminhos de destino no novo projeto

1. Preparação e Setup Inicial

Objetivo

Criar a base do novo projeto, clonar o template de exemplos e preparar o workspace para migração. Garantir que a solução .NET 8 abra corretamente e que não existam referências pendentes ao projeto antigo.

Passos que o desenvolvedor deve executar

- Clonar repositório de exemplos:

```
1 | git clone https://tfs.ons.org.br/tfs/ons/Pop/_git/Exemplos  
    <TEMP_EXAMPLES_PATH>
```

- Abrir solução no VS/VSCode: abrir `<TEMP_EXAMPLES_PATH>/ONS.Exemplo.Net.sln` .
- Rodar o prompt 1 que copia e renomeia os arquivos
- Abrir solução no VS/VSCode: abrir `<NEW_SOLUTION_PATH>/ONS.
<NEW_SYSTEM_NAME>.Net.sln` .
- Executar build inicial e anotar erros de referência.
- Abrir a ferramenta de IA, Exemplo, Amazon Q Chat, e importe o arquivo de contexto definido no item 8 deste artigo: [migration-context.md](#) 

Prompt 1 (Renomeação e cópia)

Contexto: Você é um assistente especializado em migração de projetos .NET. Temos um template de exemplo que deve ser usado como base para um novo projeto chamado `ONS.<NEW_SYSTEM_NAME>` .

Tarefa: Copie todo o conteúdo do template localizado em `<TEMP_EXAMPLES_PATH>` para `<NEW_SOLUTION_PATH>` . Renomeie nomes de arquivos, pastas e namespaces de `ONS.Exemplo` para `ONS.<NEW_SYSTEM_NAME>` . Atualize também o conteúdo de arquivos `.cs` , `.cshtml` , `.json` e quaisquer arquivos de config que contenham `ONS.Exemplo` para `ONS.<NEW_SYSTEM_NAME>` .

Regras:

1. Preserve a estrutura de pastas e arquivos; crie novos projetos onde necessário.
2. Não altere outros tokens (e.g., connection strings) -apenas namespaces e nomes de projeto.
3. Para arquivos `.cshtml` , preserve marcações Razor, atualize apenas referências a `ONS.Exemplo` .
4. Gere um relatório listando todos os arquivos alterados e as substituições realizadas.



sem erro de carregamento de projeto e sem referências a **ONS.Exemplo** após execução.

Saída esperada: Lista de arquivos alterados + instruções de follow-up.

Checklist rápido

- ☐ Template copiado para **<NEW_SOLUTION_PATH>**
- ☐ Todos os projetos carregam na IDE
- ☐ Não existem referências a **ONS.Exemplo** remanescentes (ou foram convertidas para **ONS.<NEW_SYSTEM_NAME>**)
- ☐ **launchSettings.json** contém as portas locais definidas

2. Migração de Entidades, Mapeamento EntityFramework e Mapas de Auditoria

Objetivo

Migrar entidades, enums, DTOs para o novo projeto .NET 8, adequando padrões (nullable reference types, records quando aplicável, uso de System.Text.Json, etc.)

Prompt 2 (Migrar Entidades)

Contexto: Assistente de migração. Projeto legacy em .NET 4.8 contém entidades em **<OLD_ENTITIES_PATH>** . Novo projeto em **<NEW_ENTITIES_PATH>** segue padrões do template **ONS.<NEW_SYSTEM_NAME>** .

Tarefa: Migrar todas as entidades do caminho **<OLD_ENTITIES_PATH>** para **<NEW_ENTITIES_PATH>** . Remova o decorator **DataServiceKey** e atualize namespaces e using statements. Caso uma entidade dependa de bibliotecas não compatíveis, indique a dependência e proponha alternativa (ex.: System.ComponentModel.DataAnnotations → equivalente em .NET 8).

Regras:

- Preservar nomes de propriedades e contratos públicos.
- Remova atributos/decorações obsoletas (ex.: **DataServiceKey**).
- Habilitar nullable reference types onde aplicável (adicionar **?** em propriedades opcionais).
- Ajuste namespaces para **ONS.<NEW_SYSTEM_NAME>.Domain.Entities.*** .

Validação: Compilação do projeto **ONS.<NEW_SYSTEM_NAME>.Domain.Entities** sem erros.

Saída esperada: Lista de arquivos migrados + observações sobre alterações de tipos e dependências.

Prompt 3 (Mapeamentos EF)



<NEW_MAPPING_PATH>/TipoPessoaMap.cs .

Tarefa: Migrar cada arquivo de mapeamento do diretório antigo <OLD_MAPPING_PATH> para o novo <NEW_MAPPING_PATH> , convertendo `EntityTypeConfiguration` para `IEntityTypeConfiguration` e atualizando o corpo dos métodos `Configure(EntityTypeBuilder<T> builder)` .

Registre os mapas do entity framework criado no DbContext MeuDbContext.cs

Regras:

- ▶ Utilize `builder.HasKey(...)` , `builder.Property(...)` e `builder.ToTable(...)` conforme padrões EF Core.
- ▶ Remova APIs obsoletas e substitua por equivalentes EF Core 8 quando apropriado.
- ▶ Adapte chamadas obsoletas e APIs antigas de EF para `Microsoft.EntityFrameworkCore` modernas.

Validação: `MeuDbContext` aplica os mappings e migrará com sucesso (`dotnet ef migrations add InitialMigration` deve rodar localmente sem erros em ambiente preparado).

Prompt 4 – Migrar rotinas de auditoria para o DbContext do .NET 8

Contexto: Adaptar as rotinas de auditoria do .NET Framework para a nova arquitetura no .NET 8.

Tarefa:

"Migre os métodos de Mapa de auditoria (métodos terminados em "Map") existente no DbContent antigo em .NET 4.8 para o novo DbContext no .NET 8, E registre a chamada desse métodos dentro do método MontarMapa"

Validação: Compilação do projeto `ONS.<NEW_SYSTEM_NAME>.Domain.Repos.Impl` sem erros.

Checklist Domain

- ☐ Todas as entidades migradas
- ☐ Decorator `DataServiceKey` removido ou substituído
- ☐ Mapeamentos atualizados para `IEntityTypeConfiguration`
- ☐ `MeuDbContext` registra todos os mappings

3. Migração da Camada Repository

Objetivo

Garantir que interfaces e implementações de repositório estejam compatíveis com as práticas de .NET 8 (injeção de dependência, async/await, CancellationToken, uso de IQueryable via métodos



Prompt 5 (Criar Repositórios a partir de Exemplo)

Observação: No projeto que estava sendo migrado não existia classes de repositórios no projeto antigo, então eu criei com base no existente no projeto de exemplo. Caso o seu sistema possua ajuste esse prompt para que migre o repositório antigo considerando os novos padrões.

Contexto: Repositório de exemplo contém `ITipoPessoaRepository.cs` e `TipoPessoaRepository.cs`. Diretório de entidades está em `<NEW_ENTITIES_PATH>`.

Tarefa: Usando os arquivos de exemplo, crie interfaces e implementações de repositório para todas as entidades presentes em `<NEW_ENTITIES_PATH>`.

Regras:

- Mantenha consistência de nomes e namespaces (`ONS.<NEW_SYSTEM_NAME>.Domain.Repos...`).
- Inclua comentários explicando adaptações feitas.

Validação: Projeto de Repos compila; testes unitários básicos criados para métodos CRUD.

Prompt 6 (Migrar filtros)

Observação: Caso seu projeto antigo não possua filtro ajuste o prompt para pedir que crie o filtro ao invés de migrar.

Contexto: Exemplo de classe de filtro `TipoPessoaFiltro.cs`. Diretório de filtros está em `domain\ONS.<NEW_SYSTEM_NAME>.Domain.Repos\Filtros\`.

Tarefa: Usando o arquivo de exemplo, Migre todos os filtros do projeto antigo `<PATH_FILTROS_ANTIPOS>`

Regras:

- Onde é herda de `DefaultFilter` deve ser herdado de `BaseFilterMap`
- Onde se utiliza o método `.Query()` deve ser substituído pelo método `.GetQueryable()`
- No construtor da classe deve herdar `:base()` e informar `Filter = this` assim:

```
public TipoPessoaFiltro() : base()
{
    Filter = this;
}
```
- Onde é usado `POPHelper` deve ser usada a interface `IUsuarioHelper`

Validação: Compilação do projeto `ONS.<NEW_SYSTEM_NAME>.Domain.Repos` sem erros.

Checklist Repository



- ☐ Implementações de repositórios criadas e testadas
- ☐ Filtros atualizados para `BaseFilterMap`
- ☐ Métodos `.Query()` substituídos por `.GetQueryable()`
- ☐ Repositórios registrados no container IoC

4. Migração da Camada de Negócio / Business

Objetivo

Migrar as classes de negócio para o novo projeto realizando os ajustes a nova arquitetura e criar camada de validações (Validators)

Prompt 7 (Migrar Business/Services)

Contexto: As interfaces antigas estão em `<OLD_BUSINESS_CONTRACT_PATH>` e as implementações em `<OLD_BUSINESS_IMPL_PATH>`. Exemplo de referência: `ITipoPessoaBusiness.cs` e `TipoPessoaBusiness.cs`.

Tarefa: Migrar todas as interfaces de negócio e implementações para o novo projeto, aplicando as transformações a seguir:

- Substitua `.Query()` por `.GetQueryable()`
- Substitua uso de `POPHelper` por injeção de `IUsuarioHelper`
- Substitua `common.schemas.MessageType` por `POP.Provider.Entities.Enum.MessageType`

Regras:

- Exponha métodos públicos nas interfaces de `Domain.Business.IBusiness`.
- Remova métodos duplicados após migração (mantenha somente as implementações novas).

Validação: Build do projeto `Domain.Business`.

Prompt 8 (Criar Validators)

Observação muitos dos projetos antigos não utilizam esse padrão, a adoção dessa metodologia no momento da migração é opcional, a ser definido pela equipe do projeto.

Contexto: Exemplo de referência: `TipoPessoaValidator.cs`.

Tarefa: Crie a implementação de Validators para todas as outras classes contidas em `Domain.Entities\Entities\`

Examine as classes de negócio de `Domain.Business.Impl\Business\` Identifique as regras de negócio definidas nos métodos

ValidarCriar

ValidarAtualizar



Validação: Build do projeto `Domain.Business.Impl`.

Checklist API

- ☐ Business migrados funcionalmente equivalentes
- ☐ Validators criados funcionalmente equivalentes aos métodos ValidarCriar, ValidarAtualizar, ValidarExcluir

5. Migração da Camada UI Web

Objetivo

Migrar controllers web, Razor Pages, assets estáticos (HTML/JS/CSS) e templates AngularJS para a nova estrutura `wwwroot` do projeto [ASP.NET](#)  Core.

Passos a serem executados

1. **Controllers Web:**
2. **Páginas / Views:**
3. **Help Pages:** copiar conteúdo de help para `wwwroot/HelpPages` e corrigir links.
4. **Registro de recursos:** atualize `bundleconfig.json`, `libman.json` e referências de layout para apontar para a nova estrutura.

Checklist UI

- ☐ Controllers web migrados
- ☐ Todas as páginas e assets copiados para `wwwroot`
- ☐ Caminhos relativos corrigidos nos `.js` e `.cshtml`
- ☐ Help pages migradas

Prompt 9 (Migrar Controllers Web)

Contexto: Arquitetura: controllers antigos em `<OLD_UI_CONTROLLERS_PATH>`.

Tarefa: Migre os métodos existentes em todos os controllers `.cs` antigos para novos controllers com o mesmo nome em `<NEW_UI_CONTROLLERS_PATH>`. Substitua `.Query()` por `.GetQueryable()` e `POPHelper/IHelperPOP` por `IUsuarioHelper`. Garanta que os endpoints usados pelas páginas continuem funcionais.

Regras:

- Preserve rota e signatures públicas.
- Inclua comentários sobre alterações significativas.

Validação: Projeto `ONS.<NEW_SYSTEM_NAME>.Web` compilando



Observação: Troque "Ambiente" por uma magina do seu sistema que queira migrar.

Contexto: Páginas antigas `.html` em `<OLD_UI_VIEWS_PATH>/Ambiente/` e arquivos JS/CSS associados. Exemplo de página migrada: `Pages/TipoPessoa/Index.cshtml` .

Tarefa: Mapear todos os arquivos registados na página antiga e migrar conteúdo para `Pages/Ambiente/Index.cshtml` . Copiar todos os `.html` para `wwwroot/views/Ambiente` (exceto o arquivo principal que vira Razor Page), `.js` para `wwwroot/js/Ambiente` e `.css` para `wwwroot/css/Ambiente` , mantendo subpastas. Atualize caminhos relativos nos arquivos `.js` para apontarem às novas localizações.

Regras:

- Preserve funcionalidade dos scripts (event handlers, chamadas AJAX).
- Atualize referências de URLs que apontavam para controllers antigos (se necessário).

Prompt 10.1, 10.2, 10.3, etc (Migrar outras paginas)

Observação: O prompt anterior migrou a primeira pagina do seu sistema, agora vc precisa descidir se irá manter que migre todas as outras paginas de uma vez, ou se irá migra uma por vez. Ex.:

Tarefa 10.1: Realize os mesmos passos para migrar a pagina

`<OLD_UI_VIEWS_PATH>\AmbientePacote\AmbientePacoteLista.html`

Tarefa 10.2: Realize os mesmos passos para migrar a pagina

`<OLD_UI_VIEWS_PATH>\AmbienteParametro\AmbienteParametroLista.html`

Tarefa 10.x: Realize os mesmos...

Ou crie mande migrar todas as paginas de uma vez:

Tarefa: Realize os mesmos passos para migrar todas as paginas `<OLD_UI_VIEWS_PATH>\<Pagina>`

6. Remoções e Limpeza

Objetivo

Remover classes de exemplo (Pessoa, TipoPessoa) e qualquer resquício de código de teste que não pertença à base real do projeto.

Passos executáveis

- Remover todas as classes, controllers, views e repositórios específicos de `Pessoa` e `TipoPessoa` do novo projeto.
- Rodar `grep /search` por símbolos `Pessoa` e `TipoPessoa` e revisar ocorrências.
- Atualizar documentação interna e CHANGELOG.

Prompt 11 (Remover Classes Exemplo)



final.

Tarefa: Remova todas as classes específicas dessas entidades de todos os projetos do novo repositório e reporte os arquivos removidos.

Validação: `grep -R "TipoPessoa" <NEW_SOLUTION_PATH>` não retorna arquivos relevantes além de documentação intencional.

7. Checklist Final e Pós-Migração

Checklist Operacional

- ☐ Build completo da solução passa sem erros
- ☐ HealthChecks reportam OK localmente
- ☐ OpenTelemetry está coletando traces/métricas básicos
- ☐ Configuração SSO/Keycloak valida tokens e roles
- ☐ Kafka producers/consumers testados em ambiente de staging
- ☐ Documentação atualizada (README, [CONFIGURACAO-AMBIENTE.md](#) )

Checklist por função

Dev: testes unitários, ajustes de path, revisão de performance local.

Arquiteto: validação de DI lifetimes, revisão de padrões de caching/DB, revisão de telemetria.

PO/PM: validação funcional, validação de permissões e workflows em staging.

8. Arquivo `migration-context.md` (conteúdo sugerido para alimentar Amazon Q / Copilot)



Este bloco é o conteúdo recomendado para criar um arquivo `migration-context.md` que será usado para alimentar chats/assistentes IA com o contexto completo do projeto.

```

1  # Contexto de Migração - ONS ConfigIT
2  Contexto técnico:
3  - Stack: .NET 8, EF Core, OpenTelemetry, Keycloak OpenID Connect, Kafka,
4  Horusec (SAST/DAST)
5  - Padrões: IoC por construtor, DTOs/Services/Repositories, HealthChecks,
6  appsettings.json + .env template
7  - Convenções de nomes: namespaces iniciam com ONS.ConfigIT.*, projects
8  prefixados com ONS.ConfigIT.*
9  Paths relevantes (substituir placeholders):
10 - Old entities: <OLD_ENTITIES_PATH>
11 - Old mapping: <OLD_MAPPING_PATH>
12 - Old business contract: <OLD_BUSINESS_CONTRACT_PATH>
13 - New project root: <NEW_SOLUTION_PATH>

```



```

~ - Substituir `POPHelper` por `IUsuarioHelper`
17 - Substituir `common.schemas.MessageType` por
18 `POP.Provider.Entities.Enum.MessageType`
19 - Mapear EntityTypeConfiguration → IEntityTypeConfiguration
20 - Remover DataServiceKey decorator
21 - Atualizar assets para wwwroot e ajustar caminhos relativos
22 - Validar IoC e registros em ServiceRegister.cs
23
24 ## Visão Geral do Projeto
25
26 Este documento contém o contexto completo de migração, de uma arquitetura
27 legada para .NET Core com Razor Pages.
28
29 ### Estrutura do Projeto
30
31 **Projeto Legado:**
32 - Tecnologia: HTML standalone com AngularJS
33 - Estrutura: Views organizadas por módulo em pastas separadas
34
35 **Projeto Novo:**
36 - Tecnologia: .NET Core com Razor Pages + AngularJS
37 - Estrutura: Razor Pages com arquivos estáticos organizados em wwwroot
38
39 ## Padrão de Migração Estabelecido
40
41 ### Estrutura de Arquivos do projeto web
42
43 Para cada módulo migrado, a estrutura segue o padrão:
44
45 ...
46 Pages/[Modulo]/
47 |— Index.cshtml (página principal)
48 |— Index.cshtml.cs (code-behind)
49
50 wwwroot/
51 |— css/[Modulo]/
52 |   |— [Modulo].css
53 |— js/[Modulo]/
54 |   |— [Modulo]Controller.js
55 |   |— [Modulo]ListaController.js
56 |— views/[Modulo]/
57 |   |— [Modulo].html (modal de edição)
58 ...
59
60 ### Template da Página Principal (Index.cshtml)
61
62 ````csharp
63 @page
64 @model ONS.ConfigIT.Web.Pages.[Modulo].IndexModel
65

```



```

68     }
69
70     @section Styles {
71         <link rel="stylesheet" type="text/css"
72 href="~/css/[Modulo]/[Modulo].css">
73     }
74
75     @section Scripts {
76         <script type="text/javascript"
77 src="~/js/[Modulo]/[Modulo]ListaController.js"></script>
78         <script type="text/javascript"
79 src="~/js/[Modulo]/[Modulo]Controller.js"></script>
80     }
81     <div ng-controller="[Modulo]ListaController">
82         <!-- Conteúdo da página -->
83     </div>
84     ...
85
86     ### Template do Code-Behind (Index.cshtml.cs)
87
88     ```csharp
89     using Microsoft.AspNetCore.Authorization;
90     using Microsoft.AspNetCore.Mvc;
91     using Microsoft.AspNetCore.Mvc.RazorPages;
92
93     namespace ONS.ConfigIT.Web.Pages.[Modulo]
94     {
95         [Authorize]
96         public class IndexModel : PageModel
97         {
98             public void OnGet()
99             {
100             }
101         }
102     }
103     ...
104
105     ### Validators (FluentValidation)
106     Todos os validators seguem o padrão com RuleSets por operação:
107     - `TipoPessoaValidator.cs`
108
109     ### Arquivos IoC
110     Todos seguem padrão Repository → Business → Validator:
111     - `TipoPessoaIoC.cs`
112     - Registrado em `ServiceRegister.cs`
113
114     ### DTOs
115     - `PessoaDTO.cs` - Migrado para `domain\ONS.Exemplo.Domain.Entities\DTO\`
116

```



```

119
120 - **.NET Core:** Razor Pages
121 - **Frontend:** AngularJS (mantido da versão legada)
122 - **Grid:** ag-Grid
123 - **UI:** Bootstrap + Glyphicons
124 - **Autenticação:** Keycloak via OpenID Connect
125
126 ## Comandos para Continuar Migração
127
128 Para migrar um novo módulo, siga os passos:
129
130 1. **Criar estrutura de pastas:**
131   ```bash
132   mkdir "Pages\[Modulo]"
133   mkdir "wwwroot\views\[Modulo]"
134   mkdir "wwwroot\js\[Modulo]"
135   mkdir "wwwroot\css\[Modulo]"
136   ```
137
138 2. **Migrar arquivos seguindo o padrão estabelecido**
139 3. **Corrigir caminhos dos templates nos controllers JavaScript**
140 4. **Verificar se o controller API existe e está funcional**

```

Observações Importantes

- Manter funcionalidades específicas de cada módulo
- Preservar validações e controles de segurança
- Corrigir caminhos relativos nos arquivos JavaScript
- Seguir o padrão de nomenclatura estabelecido
- Todos os módulos usam `[Authorize]` para controle de acesso
- Grid utiliza `ag-ons` como classe CSS padrão

```

1 # migration-context.md
2 Projeto: ONS.ConfigIT
3 Propósito: Fornecer contexto para assistente de migração da base legacy .NET
4 4.8 para .NET 8.
5 Contexto técnico:
6 - Projeto alvo: ONS.ConfigIT (template interno)
7 - Stack: .NET 8, EF Core, OpenTelemetry, Keycloak OpenID Connect, Kafka,
8 Horusec (SAST/DAST)
9 - Padrões: IoC por construtor, DTOs/Services/Repositories, HealthChecks,
10 appsettings.json + .env template
11 - Convenções de nomes: namespaces iniciam com ONS.ConfigIT.*, projects
12 prefixados com ONS.ConfigIT.*
13 Paths relevantes (substituir placeholders):
14 - Old entities: <OLD_ENTITIES_PATH>
15 - Old mapping: <OLD_MAPPING_PATH>
16 - Old business contract: <OLD_BUSINESS_CONTRACT_PATH>

```



```

19 - Substituir `.Query()` por `.GetQueryable()`
20 - Substituir `POPHelper` por `IUsuarioHelper`
21 - Substituir `common.schemas.MessageType` por
22 `POP.Provider.Entities.Enum.MessageType`
    - Mapear EntityTypeConfiguration → IEntityConfiguration
    - Remover DataServiceKey decorator
    - Atualizar assets para wwwroot e ajustar caminhos relativos
    - Validar IoC e registros em ServiceRegister.cs
Use este contexto quando iniciar chat com Amazon Q/Copilot -inclua trechos
de código específicos quando pedir mudanças de implementação.

```

9. PowerPrompt -automação para migração de múltiplos módulos

Uso: executar este prompt em um ambiente de IA que aceite prompts de execução repetida (Copilot/Agent/Automation). Ele instrui a IA a iterar sobre um conjunto de módulos e aplicar transformações padrão.

Prompt Power (Modelo 2 -Estruturado):

```

1 Contexto: Você é um assistente de migração .NET trabalhando em lote. Temos
2 um conjunto de módulos antigos em <OLD_MODULES_LIST_PATH>. Para cada módulo
3 M em que exista:
4 - Entidade(s) em <OLD_ENTITIES_PATH>/M
5 - Mapas em <OLD_MAPPING_PATH>/M
6 - Repositórios/Business em <OLD_BUSINESS_PATH>/M
7 Tarefa: Para cada módulo M, execute:
8 1) Copiar entidades para <NEW_ENTITIES_PATH>/M e aplicar Prompt 2 (Migrar
9 Entidades).
10 2) Copiar mapas para <NEW_MAPPING_PATH>/M e aplicar Prompt 3 (Mapeamentos
11 EF).
12 3) Criar repositórios (Prompt 6) e business (Prompt 7).
13 4) Criar Validators, IoC e registrar no ServiceRegister.
14 Regras comuns:
15 - Substituir .Query() por .GetQueryable()
    - Substituir POPHelper por IUsuarioHelper
    - Registrar mudanças em um relatório consolidado com o status por módulo
      (Sucesso/Erro/Observações)
Validação: Para cada módulo, gerar um resumo (status + arquivos alterados +
comandos para correr testes).
Saída esperada: Relatório JSON com status por módulo e lista de ações
executadas.

```

10. Troubleshooting: erros comuns e como corrigir

Erro: `operator does not exist: bigint > timestamp with time zone`



Solução: converter explicitamente o tipo ou ajustar a coluna para timestamp; ex.: `event_time > (now() - interval '1 day')::timestamp` ou garantir que a coluna seja `timestamp with time zone`.

Erro: referências antigas a `EntityTypeConfiguration` após migração

Causa: arquivos não atualizados para EF Core.

Solução: substituir implementações por `IEntityTypeConfiguration<T>` e implementar `void Configure(EntityTypeBuilder<T> builder)`.

Erro: `ERR_HTTP2_PROTOCOL_ERROR` em auth callback (/signin-oidc)

Causa: configurações de TLS/HTTP2 ou rota incorreta no `launchSettings.json` / Kestrel.

Solução: Verifique `launchSettings.json`, port/urls do Keycloak client, e desabilite HTTP/2 temporariamente para debug (ou ajustar Kestrel) e testar.

Erro: páginas não encontram assets após migração

Causa: caminhos relativos não atualizados para `wwwroot`.

Solução: atualizar referências em `.cshtml` / `.js` para `~/js/...` / `~/views/...` ou usar TagHelpers (`<script src="~/js/...">`).

Erro: testes falham por diferença de timezone/datetime

Solução: padronize uso de `DateTimeOffset` em entidades e normalize em testes (forneça cultura/UTC mocks).

Contato e Referências

- ▶ Autor: [Cláudio Branco](#) 
- ▶ [Guia de Migração .NET 4.8 → .NET 8 \(com suporte de IA\)](#)

Distribuído por [Wiki.js](#)